

Communication, Time, and Ordering

Carl Borngrund

September 3, 2026



System Model

To be able to *reason* about distributed systems we first have to state our assumptions

Real networks are messy and varied, whereas a *system model*, is an agreed-upon simplification we reason within

What guarantees we build depend entirely on which model we assume

Realistic Networks are Messy

Operational realities that are difficult to predict

Our system model cannot capture everything

For example: Shark biting fibre optic cables



System Model: Network Behaviour

A system is N processes, each running a local algorithm, with no shared memory. The systems only interact by passing messages. Our link model can thus be:

- Reliable (perfect) links

A message is received if and only if it is sent. Messages may be reordered.

- Fair-loss links

Messages may be lost, duplicated, or reordered. If we keep retrying, a message will eventually get through

- Arbitrary links

A malicious adversary may interfere with messages (eavesdrop, modify, drop, spoof, replay)

System Model: Failure Behaviour

Each node that executes a specified algorithm assumes one of the following:

- Crash-stop (fail-stop)

A node is faulty if it crashes. After crashing, it stops executing forever.

- Crash-recovery (fail-recovery)

A node may crash at any time, losing its in-memory state. It may resume executing sometime later

- Omission

Node keeps executing, but silently drops some message that should have been sent/received

- Byzantine (fail-arbitrary)

A node is faulty if it deviates from the algorithm. Faulty nodes may do anything

System Model: Timing / Synchrony Behaviour

Assume one of the following for the network and nodes:

- Synchronous

Message latency no greater than a known upper bound. Nodes execute the algorithm at a known speed

- Partially Synchronous

The system is asynchronous for some finite periods of time, synchronous otherwise

- Asynchronous

Messages can be delayed arbitrarily. Nodes can pause execution arbitrarily. No timing guarantees at all

Note: synchronisation is used in different ways in other domains within CS

Why This Matters

For each of the three parts, pick one to reason about distributed systems:

- Network:
reliable, fair-loss, or arbitrary
- Nodes:
crash-stop, crash-recovery, or Byzantine
- Timing:
synchronous, partially synchronous, asynchronous

This is the basis for any distributed algorithm (and algorithms in general)

If your assumptions are wrong, all bets are off!

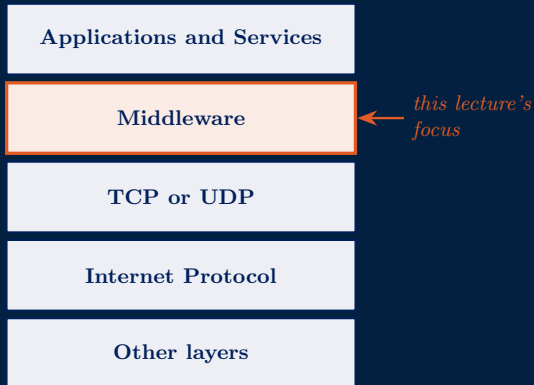
Network Protocols

Applications, services, and **middleware** lie on top of other layers that together make up the network protocol

TCP, UDP, IP, all the things you know

Applications and services describe the high-level communication between hosts

This lecture will focus on the **middleware layer**



Middleware Layer

Middleware offers an infrastructure that enables application processes to communicate with each other

Processes issue requests to the transportation layer

- (i.e. the application takes the initiative, not the middleware)

Applications access the middleware via APIs, e.g., creation and manipulation of sockets

Integration into programming languages

- remote procedure call (RPC)

For higher-level APIs, data has to be transformed before it can be transferred ("data marshalling")

- Protocols for client/server interaction ("Request/reply")

What is Interprocess Communication?

Interprocess communication (IPC) is the sharing of data between running *processes* in a computer system

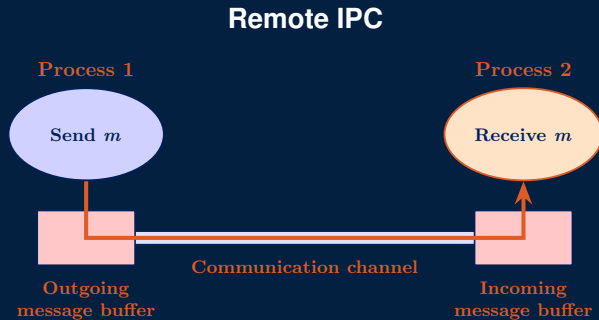
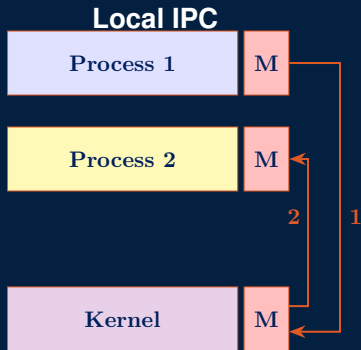
Can be seen as a client-server architecture

- In distributed systems, the applications are both

Applications:

- Operating systems
- Distributed systems
- Kernels
- ...

Local vs. Remote IPC



Characteristics of Remote IPC

IPC facilitates two operations

- **Send**(*message*)
- **Receive**(*message*)

If processes 1 and 2 wish to communicate, they need to:

1. Establish a communication channel
2. Exchange the message



Remote IPC

Local uses shared memory as it is very efficient

Shared memory is not possible over a network due to synchronisation issues and no direct access to the same physical memory

How can we get messages across?

- Efficiency matters
- Simplicity matters

Steps:

1. Package the data for transmission
2. Transmit
3. Unpack the data for use

Remote IPC

Local uses shared memory as it is very efficient

Shared memory is not possible over a network due to synchronisation issues and no direct access to the same physical memory

How can we get messages across?

- Efficiency matters
- Simplicity matters

Steps:

1. **Package the data for transmission**
2. Transmit
3. **Unpack the data for use**

Marshalling and Unmarshalling

Marshalling: Converting data or objects into a compact, transferable format (e.g., binary stream)

Unmarshalling: Reconstructing the original data or objects from the transferred format

Crucial for efficient communication in Distributed Systems

Examples:

- Protobuf
- Cap'n proto

Marshalling and Unmarshalling

Marshalling: Converting data or objects into a compact, transferable format (e.g., binary stream)

Unmarshalling: Reconstructing the original data or objects from the transferred format

Crucial for efficient communication in Distributed Systems

Examples:

- **Protobuf**
- Cap'n proto

Protobuf

Developed by Google for efficient data marshalling

Converts structured data into a compact binary format

Language-neutral, platform-neutral

Features:

1. Small message size
2. Fast serialisation/deserialisation
3. Strongly-typed data with schemas
4. Backward and forward compatibility

Protobuf: Example

Describe

```
syntax = "proto3";  
  
package example;  
  
message Person {  
    string name = 1;  
    int32 age = 2;  
}
```



Generate

```
class Person : public Message {  
public:  
    void set_name(const string& v);  
    const string& name() const;  
  
    void set_age(int32_t v);  
    int32_t age() const;  
};
```



Use

```
int main() {  
    example::Person p;  
    p.set_name("Alice");  
    p.set_age(30);  
  
    // Serialize to std::string  
    std::string data;  
    p.SerializeToString(&data);  
  
    // Deserialize from std::string  
    example::Person p2;  
    p2.ParseFromString(data);  
  
    std::cout << "Decoded Person: "  
              << p2.name() << " "  
              << p2.age() << "\n";  
    return 0;  
}
```

Remote Procedure Call

We now know how to send arbitrary data in a distributed system

- Sockets (from the network course)
- Marshalling/unmarshalling

But how do we go beyond data transfer?

How can we invoke processes or functions to run across multiple systems?

- **Remote Procedure Call**

What is RPC?

RPC allows a program to execute a procedure on machines not part of the same address space

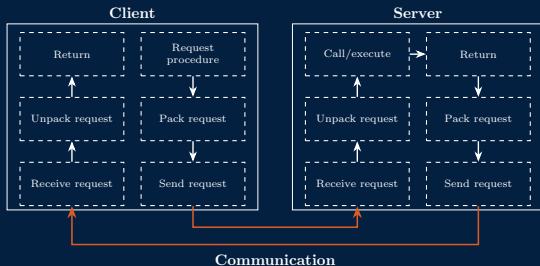
Hides the complexity of:

- Network communication
- Data serialisation
- Transport protocols

Makes distributed systems appear more like a single logical system.

Basic RPC Workflow

1. Client calls a local stub function
2. Stub handles marshalling of arguments
3. Request sent over the network
4. Remote server receives
5. Remote unmarshals
6. Remote invokes the actual function
7. Result is marshalled and sent back
8. Client receives the result like a local call



Implementations of RPC

Examples:

- Java RMI (Remote Method Invocation)
- gRPC
- JSON-RPC
- XML-RPC
- CORBA (Common Object Request Broker Architecture)
- Etc..

You implement a set of RPCs in the lab (PING, STORE, FIND_NODE, FIND_VALUE), without any library/package

Why does RPC Matter?

Simplifies building distributed applications

Enables:

- Microservices communication
- Distributed databases
- Networked games
- Remote device management

Challenges:

- Latency
- Partial failures
- Version compatibility

Why Ordering is Hard

One-to-one communication makes ordering *almost* free:

- Single sender means one source of truth for the order
- An ordered channel, such as TCP, delivers the messages in the exact order they were sent
- Request/reply is by definition sequential

Distributed systems are typically many nodes with no shared memory and *no shared clocks*

- Multiple senders → no single source of truth for order
- No shared channel → each node sees messages in a different order
- Nodes can disagree on what happened first

However, before we can reason regarding group communication, we first need to reason about order *without* a shared clock

Clocks and Time in Distributed Systems

Distributed systems often need to measure time:

- Schedulers, timeouts, failure detectors, retry timers
- Performance measurements, statistics, profiling
- Log files & databases: record when an event occurred
- Data with time-limited validity
- Determining orders of events across nodes

We often consider two types of clocks:

- **physical clocks**: count number of seconds elapse
- **logical clocks**: count events, e.g. message sent

We distinguish between clocks in a CPU (as you know from digital design) and clocks in distributed systems

Physical Clocks

Physical clocks measure time in seconds

Time comes in different formats: UTC, Unix time, etc

Time in seconds comes with difficulties such as leap seconds

Clock synchronisation is required because of *drift*

Quartz Clocks

One way is to leverage quartz clocks

- found in watches, electronics, etc

Quartz crystal that is laser-trimmed to resonate at a specific frequency

The *piezoelectrical* properties of quartz are useful as a standard of frequency

Count the number of cycles to measure time



Error: Drifting

One clock runs faster, one runs slower

Drift is measured in parts per million (ppm)

1 ppm = 1 microsecond/second = 86ms/day = 32s/year

Most computer clocks are within 50ppm

Temperature affects drift.

- $0^{\circ}C \approx -20ppm$
- $70^{\circ}C \approx -90ppm$
- $25^{\circ}C \approx 0ppm$

Atomic Clocks

When greater accuracy is required, we can use atomic clocks

Historically seconds were defined as $\frac{1}{86400}$ of a rotation of the earth

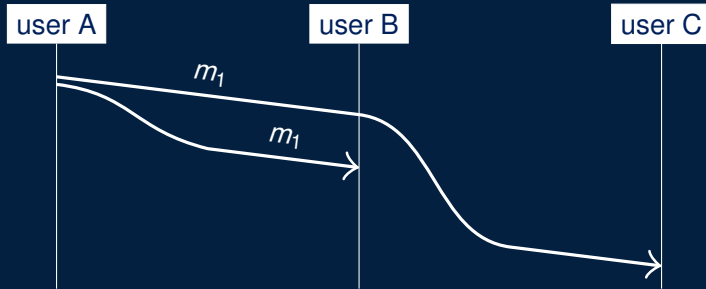
Now SI uses 1 second = 9192631770 periods of the Cs-133 resonance frequency
Accuracy ca 1 in 10^{14} (1s in every 3 million years)

Price: ca. €40000

There are other ways to keep time, but you get the idea from these two

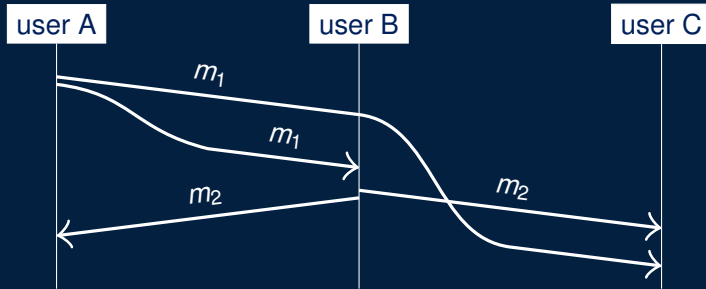


Ordering of Messages



m_1 = "A says: The moon is made of cheese!"

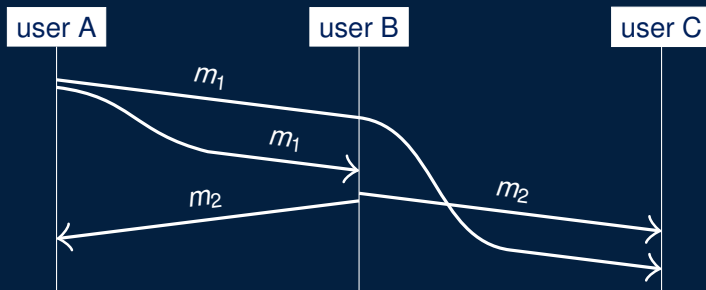
Ordering of Messages



m_1 = "A says: The moon is made of cheese!"

m_2 = "B says: Oh no it isn't!"

Ordering of Messages

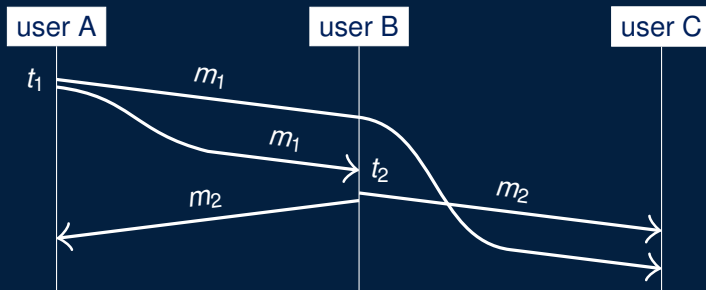


m_1 = "A says: The moon is made of cheese!"

m_2 = "B says: Oh no it isn't!"

C sees m_2 first, m_1 second, even though logically m_1 **happened before** m_2 .

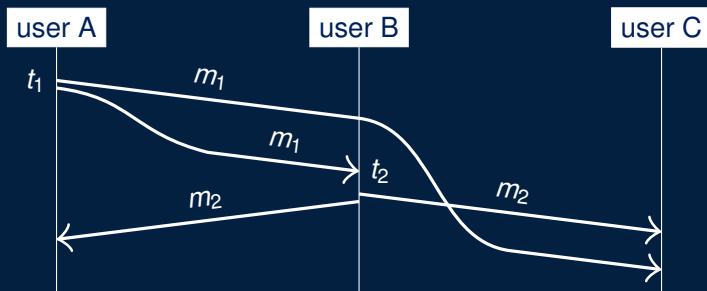
Physical Timestamps Can Lead to Inconsistencies



$m_1 = (t_1, \text{"A says: The moon is made of cheese!"})$

$m_2 = (t_2, \text{"B says: Oh no it isn't!"})$

Physical Timestamps Can Lead to Inconsistencies



$m_1 = (t_1, \text{"A says: The moon is made of cheese!"})$

$m_2 = (t_2, \text{"B says: Oh no it isn't!"})$

Problem: even with synced clocks, $t_2 < t_1$ is possible.
Timestamp order is inconsistent with expected order!

The Happens-before Relation

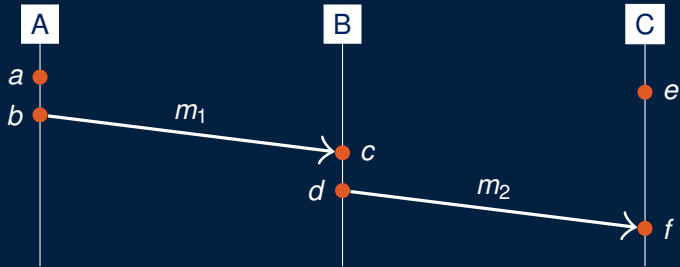
An event is something happening at one node (sending or receiving a message, or a local execution step)

We say that event a happens before event b (written $a \rightarrow b$) iff:

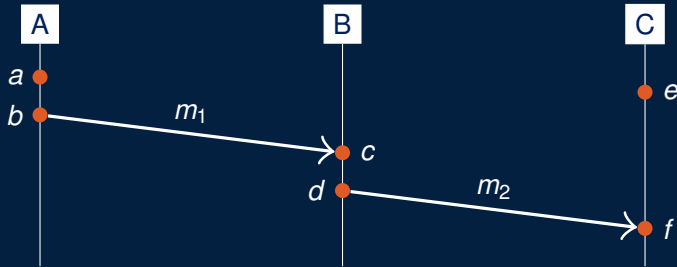
- a and b occurred at the same node, and a occurred before b in that node's local execution order; or
- event a is the sending of some message m , and event b is the receipt of that same message m (assuming sent messages are unique); or
- there exists an event c such that $a \rightarrow c$ and $c \rightarrow b$

The happens-before relation is a partial order: it is possible that neither $a \rightarrow b$ nor $b \rightarrow a$. In that case, a and b are considered *concurrent* (written $a||b$)

Happens-before Relation: Example

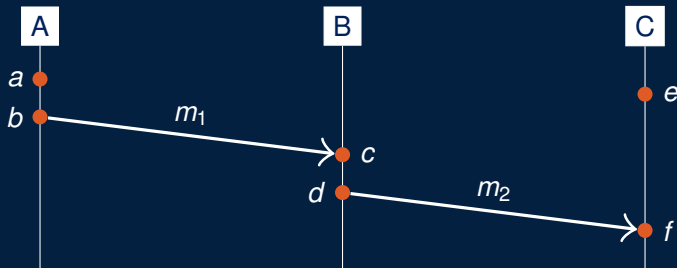


Happens-before Relation: Example



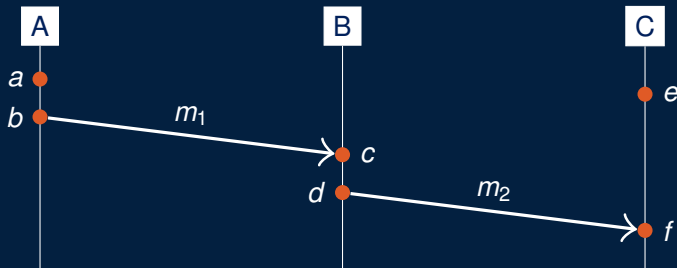
■ $a \rightarrow b$, $c \rightarrow d$, and $e \rightarrow f$ due to node execution order

Happens-before Relation: Example



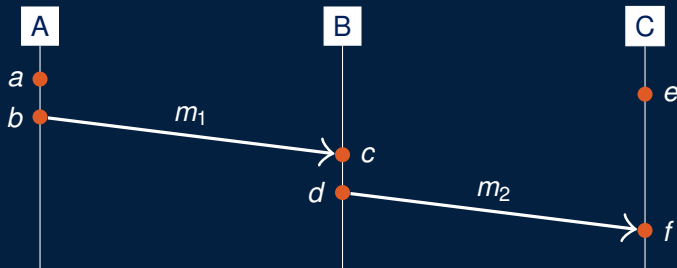
- $a \rightarrow b$, $c \rightarrow d$, and $e \rightarrow f$ due to node execution order
- $b \rightarrow c$ and $d \rightarrow f$ due to messages m_1 and m_2

Happens-before Relation: Example



- $a \rightarrow b$, $c \rightarrow d$, and $e \rightarrow f$ due to node execution order
- $b \rightarrow c$ and $d \rightarrow f$ due to messages m_1 and m_2
- $a \rightarrow c$, $a \rightarrow d$, $a \rightarrow f$, $b \rightarrow d$, $b \rightarrow f$, and $c \rightarrow f$ due to transitivity

Happens-before Relation: Example



- $a \rightarrow b$, $c \rightarrow d$, and $e \rightarrow f$ due to node execution order
- $b \rightarrow c$ and $d \rightarrow f$ due to messages m_1 and m_2
- $a \rightarrow c$, $a \rightarrow d$, $a \rightarrow f$, $b \rightarrow d$, $b \rightarrow f$, and $c \rightarrow f$ due to transitivity
- $a \parallel e$, $b \parallel e$, $c \parallel e$, and $d \parallel e$

Causality

Taken from physics (relativity)

- When $a \rightarrow b$, then a **might have caused** b .
- When $a \parallel b$, we know that a **cannot have caused** b .

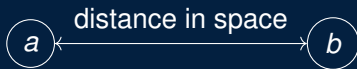
Happens-before relation encodes **potential causality**.

Causality

Taken from physics (relativity)

- When $a \rightarrow b$, then a **might have caused** b .
- When $a \parallel b$, we know that a **cannot have caused** b .

Happens-before relation encodes **potential causality**.

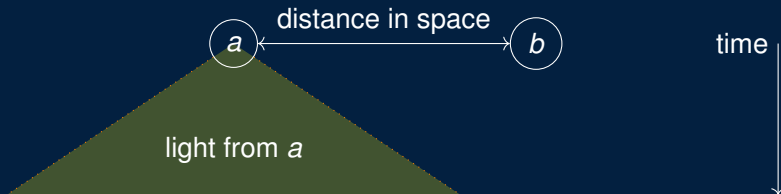


Causality

Taken from physics (relativity)

- When $a \rightarrow b$, then a **might have caused** b .
- When $a \parallel b$, we know that a **cannot have caused** b .

Happens-before relation encodes **potential causality**.



Causality

Taken from physics (relativity)

- When $a \rightarrow b$, then a **might have caused** b .
- When $a \parallel b$, we know that a **cannot have caused** b .

Happens-before relation encodes **potential causality**.

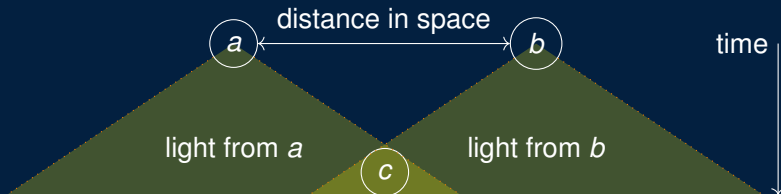


Causality

Taken from physics (relativity)

- When $a \rightarrow b$, then a **might have caused** b .
- When $a \parallel b$, we know that a **cannot have caused** b .

Happens-before relation encodes **potential causality**.

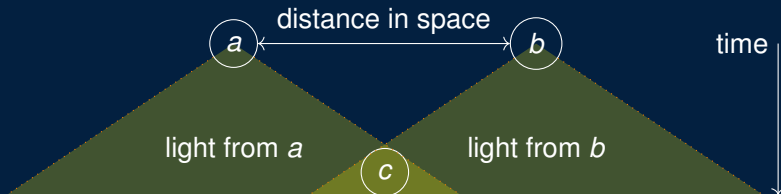


Causality

Taken from physics (relativity)

- When $a \rightarrow b$, then a **might have caused** b .
- When $a \parallel b$, we know that a **cannot have caused** b .

Happens-before relation encodes **potential causality**.



Let $\prec$ be a strict total order on events.

If $(a \rightarrow b) \implies (a \prec b)$ then $\prec$ is a **causal order**

(or: $\prec$ is “consistent with causality”)

Logical Clocks

- Physical clock: count number of **seconds elapsed**
- Logical clock: count number of **events occurred**

Physical timestamps: useful for many things, but may be **inconsistent with causality**.

Logical clocks: designed to **capture causal dependencies**.

$$(e_1 \rightarrow e_2) \implies (T(e_1) < T(e_2))$$

We will look at two types of logical clocks:

- Lamport clocks
- Vector clocks

Lamport Clocks: Algorithm

on initialisation do:

$t := 0$

▷ each node has its own local variable t

on any event occurring at the local node do:

$t := t + 1$

on request to send message m do:

$t := t + 1$; send (t, m) via the underlying network link

on receiving (t', m) via the underlying network link do:

$t := \max(t, t') + 1$

deliver m to the application

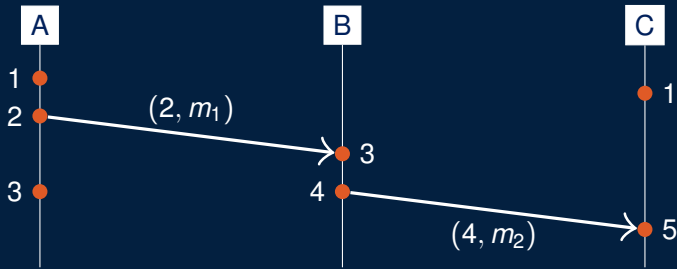
Lamport Clocks in Words

- Each node maintains a counter t , incremented on every local event e
- Let $L(e)$ be the value of t after that increment
- Attach current t to messages sent over network
- Recipient moves its clock forward to timestamp in the message (if greater than local counter), then increments

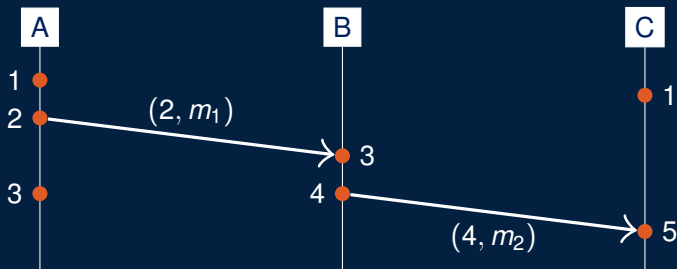
Properties of this scheme:

- If $a \rightarrow b$ then $L(a) < L(b)$
- However, $L(a) < L(b)$ does not imply $a \rightarrow b$
- Possible that $L(a) = L(b)$ for $a \neq b$

Lamport Clock: Example

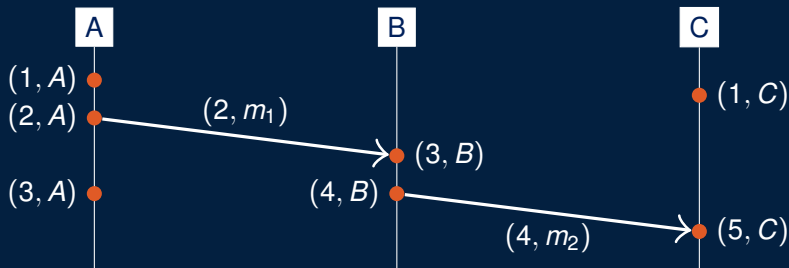


Lamport Clock: Example



Let $N(e)$ be the node at which event e occurred.
Then the pair $(L(e), N(e))$ **uniquely identifies** event e .

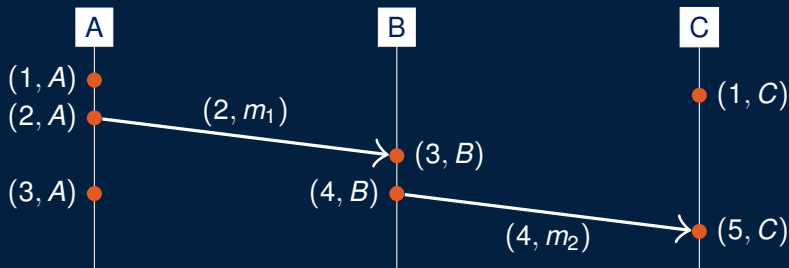
Lamport Clock: Example



Let $N(e)$ be the node at which event e occurred.

Then the pair $(L(e), N(e))$ **uniquely identifies** event e .

Lamport Clock: Example



Let $N(e)$ be the node at which event e occurred.

Then the pair $(L(e), N(e))$ **uniquely identifies** event e .

Define a **total order** $\prec$ using Lamport timestamps:

$$(a \prec b) \iff (L(a) < L(b) \vee (L(a) = L(b) \wedge N(a) < N(b)))$$

This order is **causal**: $(a \rightarrow b) \implies (a \prec b)$

Vector Clocks: Algorithm

on initialisation at node N_i do:

$T := \langle 0, 0, \dots, 0 \rangle$

▷ local variable at node N_i

on any event occurring at node N_i do:

$T[i] := T[i] + 1$

on request to send message m at node N_i do:

$T[i] := T[i] + 1$; send (T, m) via network

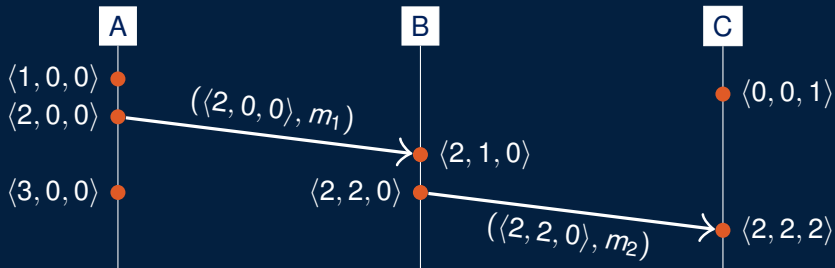
on receiving (T', m) at node N_i via the network do

$T[j] := \max(T[j], T'[j])$ for every $j \in \{0, \dots, n-1\}$

$T[i] := T[i] + 1$; deliver m to the application

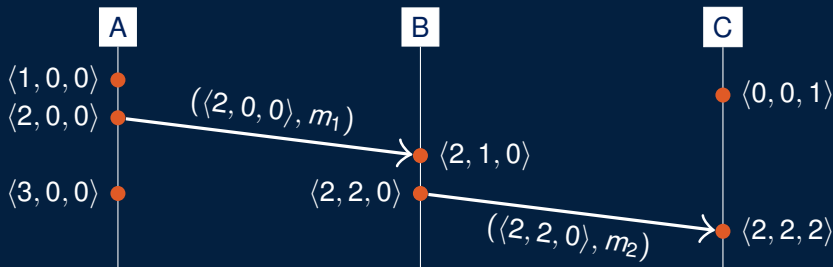
Vector Clocks: Example

Assuming the vector of nodes is $N = \langle A, B, C \rangle$:



Vector Clocks: Example

Assuming the vector of nodes is $N = \langle A, B, C \rangle$:



The vector timestamp of an event e represents a set of events, e and its causal dependencies: $\{e\} \cup \{a \mid a \rightarrow e\}$

For example, $\langle 2, 2, 0 \rangle$ represents the first two events from A, the first two events from B, and no events from C.

Direct Communication

As we discussed earlier, RPC is typically a direct communication technique

- Requests go straight from sender to receiver

Direct communication is:

- Simple and intuitive
- Synchronous by default
- Ideal for tightly-coupled systems

But direct communication isn't always ideal for distributed systems

Indirect Communication

An alternative: **Indirect communication**

- No direct coupling between sender and receiver(s)
- Messages pass through an intermediary

Space decoupling

- sender does not know the identity of receiver(s)

Time decoupling

- sender and receiver(s) have independent lifetimes:
 - they do not need to exist at the same time

Two examples: Group communication and Publish-subscribe

Broadcast (Group Communication)

Broadcast (multicast) is **group communication**:

- One node sends message, all nodes in group deliver it
- Set of group members may be fixed (static) or dynamic
- If one node is faulty, remaining group members carry on
- Note: concept is more general than IP multicast (we build upon point-to-point messaging)

Build upon system models:

- Can be **best-effort** (may drop messages) or **reliable** (non-faulty nodes deliver every message, by retransmitting dropped messages)
- Asynchronous/partially synchronous timing model
⇒ **no upper bound** on message latency

Forms of Reliable Broadcast

FIFO broadcast:

If m_1 and m_2 are broadcast by the same node, and $\text{broadcast}(m_1) \rightarrow \text{broadcast}(m_2)$, then m_1 must be delivered before m_2

Causal broadcast:

If $\text{broadcast}(m_1) \rightarrow \text{broadcast}(m_2)$ then m_1 must be delivered before m_2

Total order broadcast:

If m_1 is delivered before m_2 on one node, then m_1 must be delivered before m_2 on all nodes

FIFO-total order broadcast:

A combination of FIFO broadcast and total order broadcast

Example of Broadcast Algorithms

1. Naive Broadcast
2. Eager reliable broadcast
3. Gossip protocols
4. FIFO broadcast
5. Causal broadcast algorithm
6. Total order broadcast
7. etc..

Example of Broadcast Algorithms

1. Naive Broadcast
2. Eager reliable broadcast
3. **Gossip protocols**
4. FIFO broadcast
5. Causal broadcast algorithm
6. Total order broadcast
7. etc..

Gossip Protocol

A Gossip Protocol is a protocol that is used to spread information in peer-to-peer systems

Inspired how epidemics spread

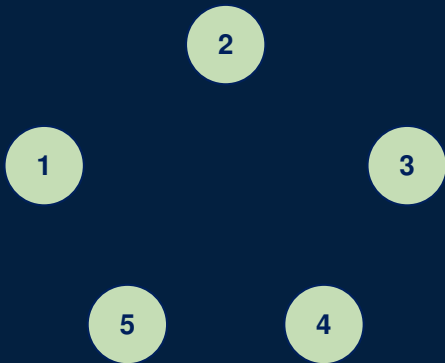
Each node periodically shares information with a few random peers

Over time, information propagates throughout the entire system, often with **high reliability**

Key Characteristics:

- No central coordinator
- Scalable to large, dynamic networks
- Resilient to node failures
- Often used for probabilistic, eventually consistent data spreading

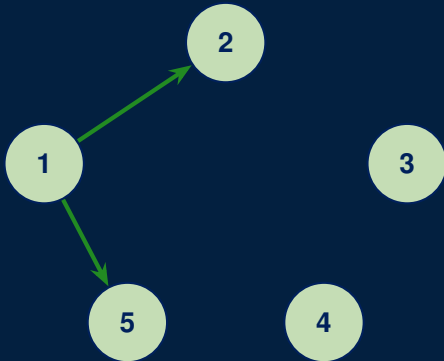
Gossip Protocol: Example



Round 0:

No information has been propagated

Gossip Protocol: Example

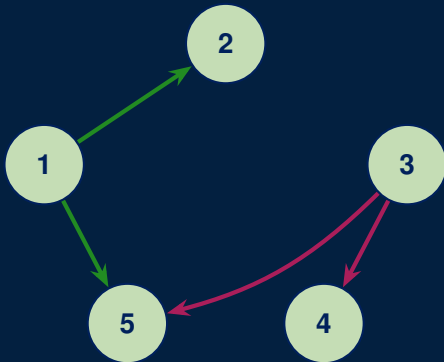


Round 1:

Node 1 propagates everything it knows to 2 and 5

Currently, node 1 only knows about itself

Gossip Protocol: Example

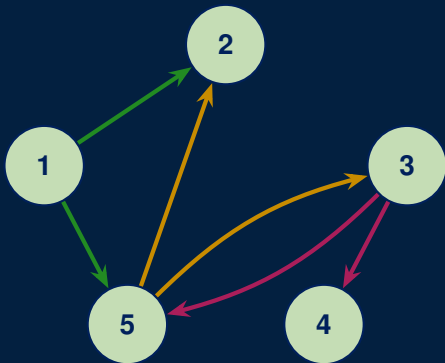


Round 2:

Node 3 propagates to 4 and 5

Currently, node 3 only knows about itself

Gossip Protocol: Example



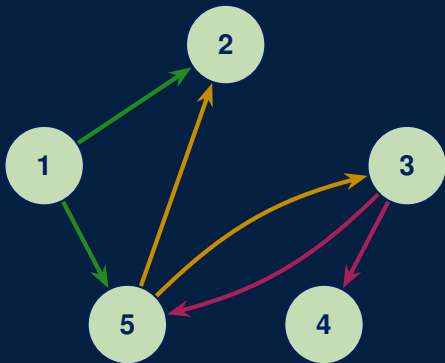
Round 3:

Node 5 propagates to 2 and 3

Since node 5 has more knowledge than itself, the communication will also include:

- Node 5 info
- Node 1 info
- Node 3 info

Gossip Protocol: Example



After time all state information will have been propagated to most nodes

State disseminated in $O(\log N)$ rounds, where N is the number of nodes

We can propagate all state information efficiently without a leader

Summary

- **Assumptions come first:** What you can guarantee depends entirely on the system model you choose
- **Communication is built in layers:** Middleware hides marshalling and the network so a remote call looks local
- **Order needs logical time, not physical time:** Without a shared clock, happens-before captures causality where physical clocks can't
- **Decoupling buys scale:** Indirect, group communication frees senders and receivers from each other's identity and lifetime

Adapted From

<https://www.ida.liu.se/~TDDD25/lectures/lect3.pdf>

<https://blogs.ischool.berkeley.edu/i206s13/files/2013/01/lec19.pdf>

<https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf>

https://www.youtube.com/watch?v=UEAMfLPZZhE&list=PLeKd45zvjcDFUEv_ohr_HdUFe97RItdiB

